Explore gradient issues

```
suppressPackageStartupMessages(library(doFuture))
library(ggplot2)
suppressPackageStartupMessages(library(rstan))
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

wd <- "~/projects/climategrowthshifts/analysis/mountrainier"

# load data for which we got init failure in first_handling.R
mdl.data <- readRDS(file.path(wd, 'output', 'saved_mdldata_initfailure.rds'))
```

Brute-force approach

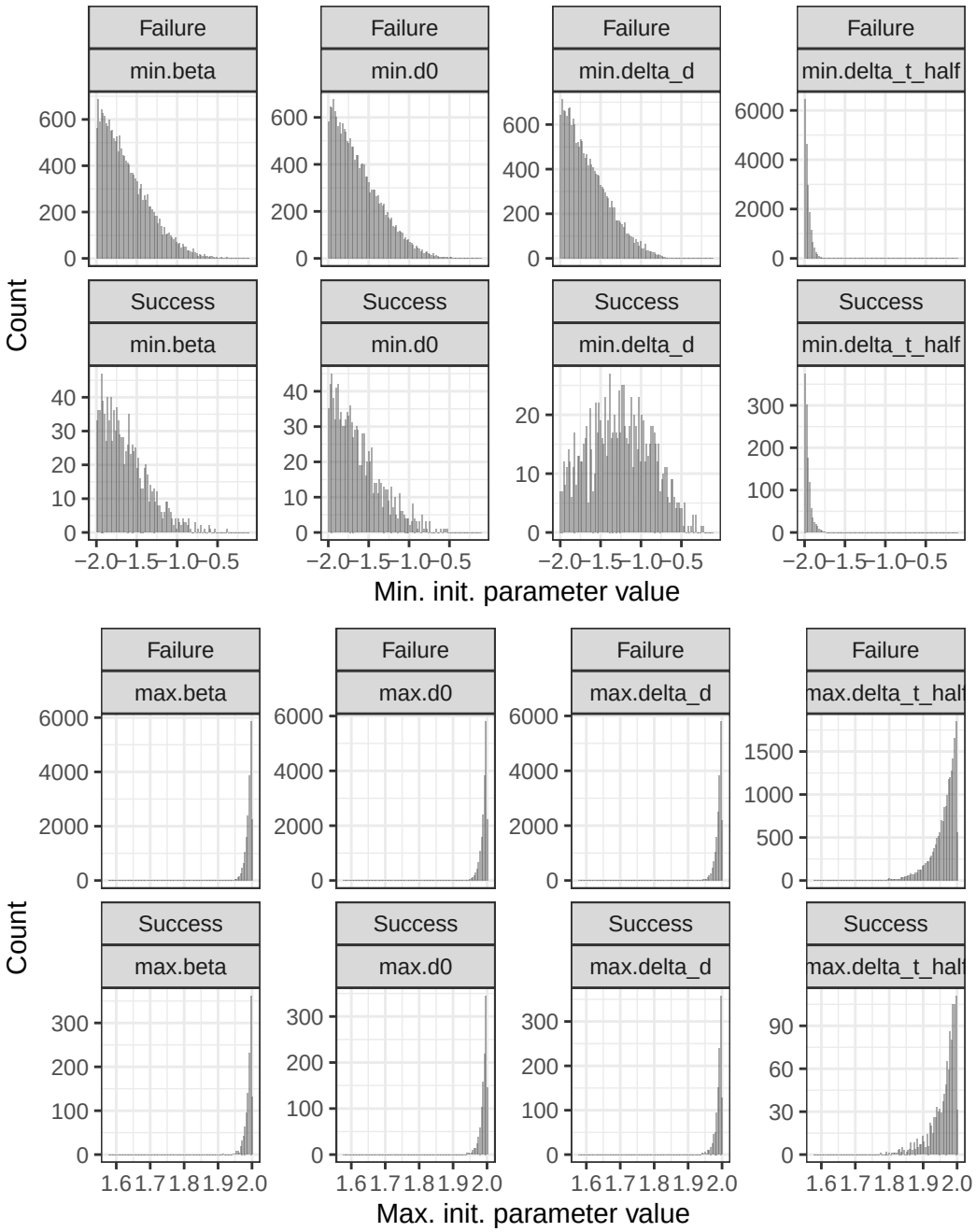
We define a function to set initial values for sampling. We draw uniformly from the interval (-2,2), the default strategy in Stan (on the unconstrained scale). For parameters that are bounded below at 0 (all except `delta_t_half`), we apply the exponential.

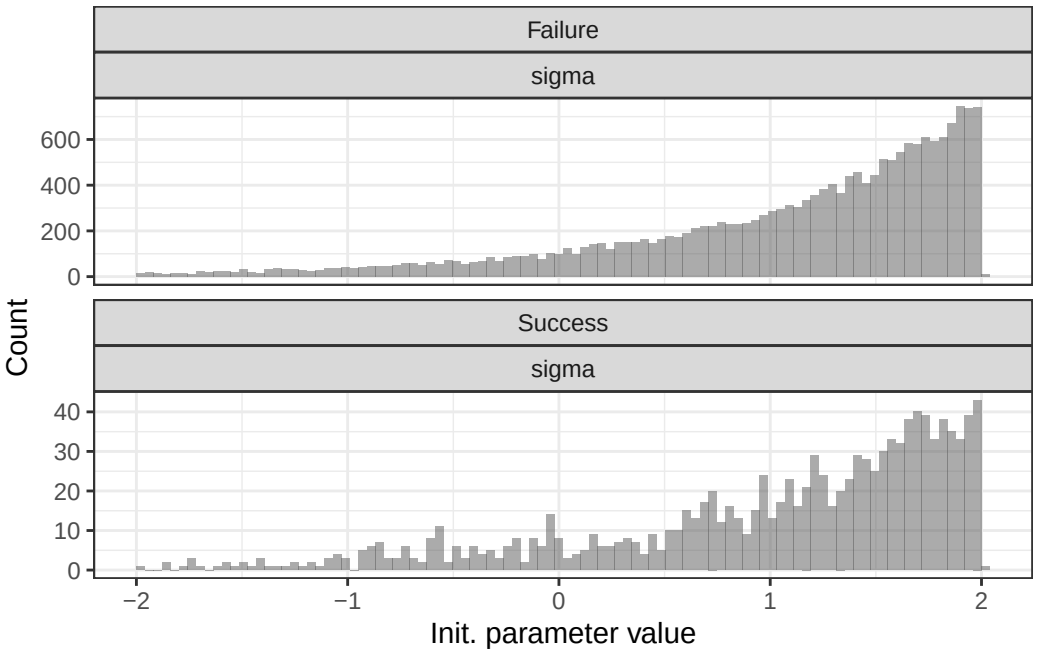
```
initfun <- function() {
  list(d0 = runif(mdl.data$N_trees,exp(-2),exp(2)),
       delta_d = runif(mdl.data$N_trees,exp(-2),exp(2)),
       beta = runif(mdl.data$N_trees,exp(-2),exp(2)),
       delta_t_half = runif(mdl.data$N_trees,-2,2),
       sigma = runif(1,exp(-2),exp(2)))
}
```

Then we can run 20000 different Stan sampling (luckily, in parallel) to explore gradient failure.

```
gompertz <- stan_model(file.path(wd, "stan/heterogeneous_gompertz.stan"))
plan(multisession, workers = 20)
init_fail <- foreach(i = 1:20000, .combine = rbind,
                     .options.future = list(seed = TRUE)) %dofuture%{
  set.seed(sample.int(.Machine$integer.max, 1))

  inits <- initfun()
  fit <-  sampling(gompertz, mdl.data,
                   chains = 1, cores = 1,
                   iter = 1, warmup = 0,
                   init = list(inits))

  data.frame(failure = ifelse(is.na(nrow(fit)), 'Failure', 'Success'),
             min.d0 = log(min(inits$d0)),
             min.delta_d = log(min(inits$delta_d)),
             min.beta = log(min(inits$beta)),
             min.delta_t_half = min(inits$delta_t_half),
             max.d0 = log(max(inits$d0)),
             max.delta_d = log(max(inits$delta_d)),
             max.beta = log(max(inits$beta)),
             max.delta_t_half = max(inits$delta_t_half),
             sigma = log(inits$sigma))

}
plan(sequential)

perc_fail <- nrow(init_fail[init_fail$failure == "Failure",])/nrow(init_fail)*100
```

Sampling fails in 94.315% of cases.
The only parameter where there is an obvious difference between “Failure” and “Success” is delta_d (the difference between minimum and maximum DBH).


Failure
min.beta
min.d0
min.delta_d
min.delta_t_half
Success
Count
Min. init. parameter value
max.beta
max.d0
max.delta_d
max.delta_t_half
Max. init. parameter value



Looking at delta_d

Taking the Gompertz function (reparameterized):

$$G(t) = d_{max} * exp(-log(2) * exp(-\frac{2}{log(2)} * \frac{\beta}{d_{max}} * (t - \tau))$$

If we write:

$$D = log(2) * exp(-\frac{2}{log(2)} * \frac{\beta}{d_{max}} * (t - \tau))$$

We have:

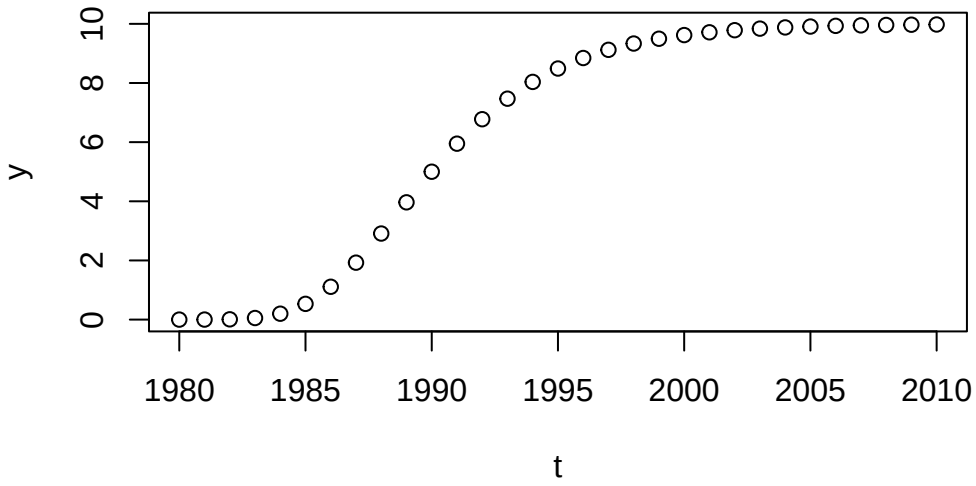
$$G(t) = d_{max} * exp(-D)$$

Then, the partial derivative with respect to dmax (`delta_d`) should be something like:

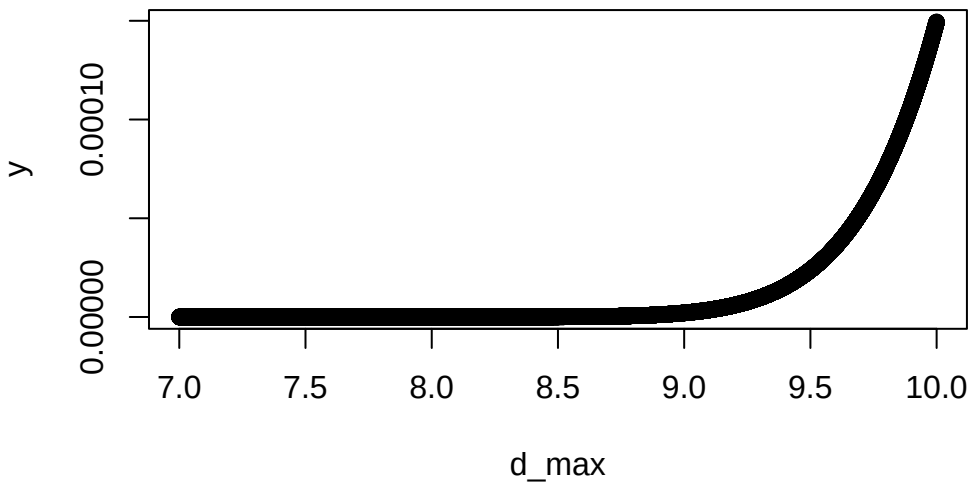
$$\frac{\partial G}{\partial d_{max}} = exp(-D)(1 - \frac{2}{log(2)}\frac{\beta}{d_{max}}(t - \tau)D)$$

```
f = expression(d_max * exp(-log(2) * exp( -   (2 / log(2)) * (beta / d_max) *
                                            (t - (2000 + delta_t_half)))))
df_ddmax <- D(f,'d_max')

d0 <- 3
beta <- 1 # growth rate
```

```r
delta_t_half <- -10
d_max <- 10

t <- 1980:2010
y <- eval(f)
plot(y ~ t)
```

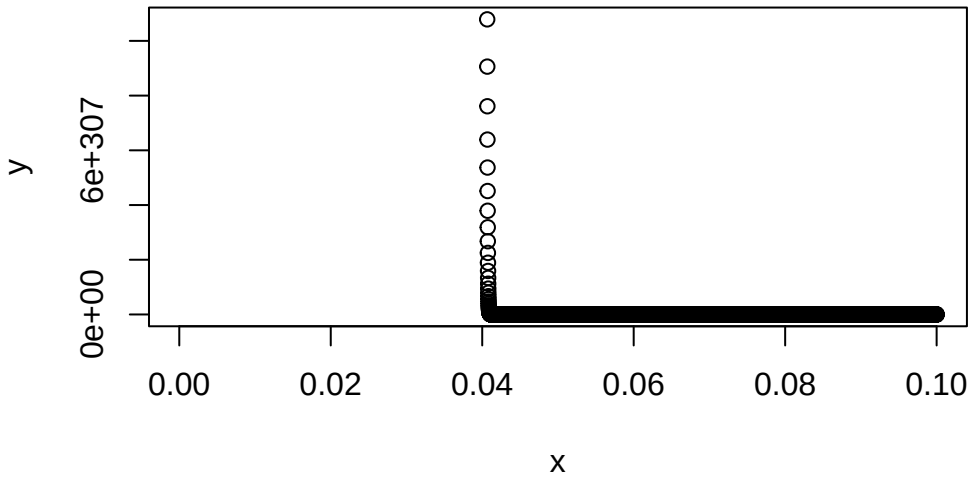



```r
d_max <- 10

t <- 1980
d_max <- seq(7,10, 0.0001)
y <- eval(df_ddmax)
plot(y ~ d_max)
```

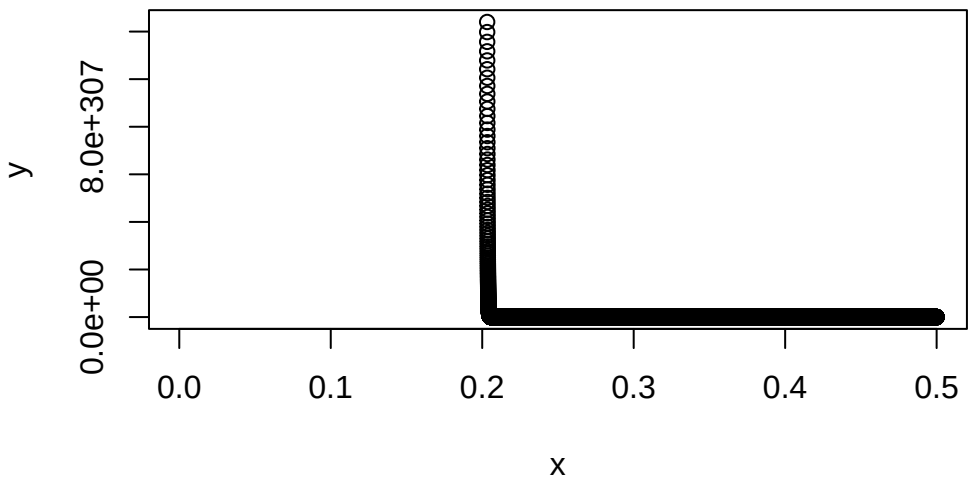


Looking at $D = log(2) * exp(-\frac{2}{log(2)} * \frac{\beta}{d_{max}} * (t - \tau))$:

```
f = expression(log(2) * exp(-(2 / log(2)) * (beta / x) *
                                (t - (2000 + delta_t_half))))
x <- seq(0,0.1,0.00001)
y <- eval(f)
plot(y ~ x)
```




```
beta <- 5
f = expression(log(2) * exp(-(2 / log(2)) * (beta / x) *
                                (t - (2000 + delta_t_half))))
x <- seq(0,0.5,0.00001)
y <- eval(f)
plot(y ~ x)
```



D quickly become very high for some values of d_{max}, at least when $(t-\tau) < 0$. In my example, $\tau = 2000 - 10$, we then get $-(2/log(2)) * (t - \tau) \simeq 28.85$, which quickly drive the exponential towards infinity.